

MMI 808 · CATEGORICAL STRUCTURES AND DIAGRAMS

MSc in Mathematical Innovation · Open University of Kenya

Module 5: What Are Databases?

Developer: Dr. George Simmons | Instructional Hours: 6

LEARNING OUTCOMES

1. Describe the abstract notion of a database schema as a categorical object — a directed graph with constraints.
2. Explain and construct example schemas using internal keys, foreign keys, and referential integrity constraints.
3. Understand the concept of data migration as a structure-preserving map between schemas (a functor).
4. Design a database schema for a real-world contact management system, representing it as both a relational schema and a categorical diagram.

1 Module Overview

Modules 1–4 built our graph-theoretic vocabulary. We now make our first decisive step into category theory by reinterpreting databases categorically. This reinterpretation — due to David Spivak — is not merely a theoretical exercise: it reveals the deep structure of databases that conventional SQL-based training leaves implicit. A database schema is a category; a database instance is a functor; data migration is a natural transformation. By the end of Module 8, this sentence will be completely clear.

WHEN IS A DATABASE MORE THAN A TABLE?

A single flat table (a spreadsheet) is the simplest possible database. But real data has *relationships*: a customer has many orders; an order has many items; an item belongs to a category. These relationships are what make databases interesting — and what a purely tabular view obscures. Category theory provides the language to make these relationships explicit, composable, and analysable.

2 Traditional Database Concepts

2.1 The Relational Model

A **relational database** organises data into **tables** (relations). Each table has:

- A set of **columns** (attributes), each with a specified data type.

- A set of **rows** (tuples), each representing one record.
- A **primary key** — a column (or combination of columns) uniquely identifying each row.
- **Foreign keys** — columns containing primary key values from another table, establishing referential links.

EXAMPLE 1: A Simple Relational Schema

A contact management database might have:

Person			Phone	
<u>person_id</u>	name	phone_id (FK → Phone)	<u>phone_id</u>	number
1	Alice	101	101	0712345678
2	Bob	102	102	0798765432

The foreign key `phone_id` in `Person` references the primary key `phone_id` in `Phone`, establishing a one-to-one relationship.

2.2 The Schema vs. the Instance

A crucial distinction:

- The **schema** defines the structure: what tables exist, what columns they have, what their types are, and what relationships connect them. The schema is fixed (or slowly evolving).
- The **instance** is the current data: the actual rows in each table at a specific time. The instance changes with every insert, update, or delete.

Categorical reading: The schema is a category; the instance is a functor from the schema category to **Set**.

3 The Categorical View of a Database Schema

SCHEMA AS A CATEGORY (Spivak)

A **database schema** in the categorical sense is a directed graph $\mathcal{S} = (Ob, Arr)$ where:

- Ob is the set of **table names** (entities).
- Arr is the set of **column names** and **foreign key references** (arrows between tables).
- Additionally, certain **path equations** hold: commutativity conditions ensuring data consistency.

Tables are objects in the schema category. **Columns pointing to a value type**

(e.g., string, integer) are arrows from the table to the type object. **Foreign keys** are arrows from one table to another.

EXAMPLE 2: Contact List Schema as a Categorical Diagram

Schema diagram for a phone contact list:

$$\begin{array}{ccccc} \text{Person} & \xrightarrow{\text{hasPhone}} & \text{Phone} & \xrightarrow{\text{number}} & \text{String} \\ & & \text{Person} & \xrightarrow{\text{name}} & \text{String} \\ & & \text{Person} & \xrightarrow{\text{worksAt}} & \text{Company} \xrightarrow{\text{companyName}} \text{String} \end{array}$$

Objects: Person, Phone, Company, String (a “value type”).

Arrows: hasPhone (Person $\rightarrow$ Phone), name (Person $\rightarrow$ String), number (Phone $\rightarrow$ String), worksAt (Person $\rightarrow$ Company), companyName (Company $\rightarrow$ String).

Path equation: Should the path Person $\xrightarrow{\text{hasPhone}}$ Phone $\xrightarrow{\text{number}}$ String be the same as some direct “personalNumber” arrow Person $\rightarrow$ String? Only if every person’s personal number is their phone’s number — a business rule enforced as a commutativity condition.

4 Key Relationships: One-to-One, One-to-Many, Many-to-Many

CARDINALITY RELATIONSHIPS

One-to-one (1:1): Each row in table A corresponds to exactly one row in table B , and vice versa. Represented by a bijective foreign key.

One-to-many (1:N): Each row in table A corresponds to zero or more rows in table B . A foreign key in B points to A ’s primary key. Categorically: a function $B \rightarrow A$ (many B rows map to one A row).

Many-to-many (M:N): Each row in A can relate to many rows in B and vice versa. Requires a **junction table** with foreign keys to both A and B .

EXAMPLE 3: Representing Relationships Categorically

Person hasPhone Phone (many-to-one: many persons may share a landline): The arrow hasPhone: Person $\rightarrow$ Phone is a function — each person maps to exactly one phone entry; multiple persons can map to the same phone.

Person studiedAt University (many-to-many): A person can study at multiple universities; a university has many students. Create a junction table **Enrollment** with foreign keys to both Person and University.

Categorical representation:

$$\text{Person} \leftarrow \text{Enrollment} \rightarrow \text{University}$$

Enrollment has arrows (foreign keys) to *both* Person and University — it is the categorical product of a many-to-many relationship.

5 Data Migration: Introducing Functors

DATA MIGRATION (Preview)

A **data migration** is a process of transforming data from one schema to another. Categorically, a migration is induced by a **functor** $F : \mathcal{S} \rightarrow \mathcal{T}$ between schema categories. Given an instance $I : \mathcal{T} \rightarrow \mathbf{Set}$ of the target schema, we can “pull back” to get an instance $I \circ F : \mathcal{S} \rightarrow \mathbf{Set}$ of the source schema — or “push forward” in the opposite direction.

We will develop the full theory of functors and data migration in Modules 7–8.

6 Wiring Diagrams

A **wiring diagram** is a graphical representation of a schema where:

- Each table is drawn as a box.
- Each column/foreign key is drawn as a wire (arrow) connecting boxes.
- Value types (String, Int, Bool) are drawn as external wires (outputs of the schema).

Wiring diagrams are a categorical notation that will become central when we study monoidal categories and database joins in advanced category theory.

7 Key Concepts and Terminology

Schema	The structure of a database: tables, columns, types, keys, and relationships.
Instance	The current data in a database: the actual rows populating each table.
Primary Key	A column (or combination) uniquely identifying each row in a table.
Foreign Key	A column in one table referencing the primary key of another — a schema arrow.
Referential Integrity	Constraint ensuring every foreign key value references an existing primary key.
Cardinality	The nature of a relationship: 1:1, 1:N, or M:N.
Categorical Schema	A directed graph (schema diagram) with path equations; objects = tables, arrows = columns/FKs.
Functor (preview)	A structure-preserving map between categories; induces data migration between schemas.
Wiring Diagram	Graphical notation for categorical schemas; boxes = tables, wires = columns/FKs.

8 Module Assignment

MODULE 5 ASSIGNMENT

Contact List Database Schema: Refine and formalise a database schema for the contact list on your phone (or a hypothetical one).

1. **Relational representation:** Define at least 4 tables with columns, primary keys, and foreign keys. Identify all cardinality relationships (1:1, 1:N, M:N). Handle M:N relationships with junction tables.
2. **Categorical diagram:** Draw the schema as a directed graph (objects = tables + value types, arrows = columns + foreign keys). Label all arrows.
3. **Path equations:** Identify at least one business rule that corresponds to a commutativity condition in your schema diagram. State the path equation explicitly.
4. **Wiring diagram:** Redraw your schema as a wiring diagram (boxes and wires).
5. **Reflection:** What information does the categorical diagram reveal about your schema that the standard table representation does not?

9 Revision Questions

1. Explain the difference between a database schema and a database instance. In categorical terms, what corresponds to each?
2. In Spivak's categorical view, a "column" of a table is an arrow. Where does this arrow point? Give two examples: one where the arrow points to a value type, and one where it points to another table.
3. Design a schema (both relational and categorical) for a university course registration system. Include: Student, Course, Lecturer, Enrollment, Department. Identify all cardinality relationships.
4. What is referential integrity? Show with an example how a missing foreign key value violates referential integrity, and describe the SQL constraint that prevents it.
5. In the contact list schema (Example 2), the path equation $\text{Person} \xrightarrow{\text{hasPhone}} \text{Phone} \xrightarrow{\text{number}} \text{String}$ might or might not equal $\text{Person} \xrightarrow{\text{personalNumber}} \text{String}$. Give a real-world scenario where it would commute and one where it would not.
6. A medical records system needs to store: patients, their doctors, their diagnoses, and the medications prescribed for each diagnosis. Design a schema handling the fact that: a patient has many doctors; a diagnosis can be given by any of the patient's doctors; a diagnosis can result in multiple medications. Draw the categorical schema diagram.
7. Preview: If a schema $\mathcal{S}$ has objects $\{A, B, C\}$ and arrows $\{f : A \rightarrow B, g : B \rightarrow C\}$, what is the free category on this schema? How many morphisms does it have?

MODULE 5 SUMMARY

- A database schema is the structure of a database — its tables, columns, types, and relationships. An instance is the current data populating that structure.
- Spivak's categorical interpretation identifies schemas with directed graphs augmented by path equations; tables are objects, columns are arrows.
- Cardinality relationships (1:1, 1:N, M:N) correspond to different categorical structures: bijections, functions, and spans respectively.
- Foreign keys are categorical arrows; referential integrity is a type constraint enforcing that every arrow points to a valid target.
- Data migration between schemas is induced by a functor — a concept developed fully in Modules 7–8.
- *When you see a database, you should now see a category: objects and arrows, composition and identity, structure-preserving maps.*

10 Further Reading

- **Fong, B. & Spivak, D.I.** (2019). *Seven Sketches in Compositionality*. Section 3.1. Cambridge University Press.
- **Spivak, D.I.** Category Theory for Scientists. Lecture 6: Databases. <https://arxiv.org/abs/1302.6946>

- **Codd, E.F.** (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387. — Original relational model paper.